Bizzdesign Data Flow Scripts: User Manual

This guide covers installing the scripts in Bizzdesign Enterprise Studio, choosing the correct output presentation, and using the scripts safely. The scripts are stored in the model as script viewpoints; they are not installed as a separate plugin.

1. Install the scripts

Install the scripts in each model where they are needed. A saved script viewpoint belongs to that model.

1. Make a backup or work in a disposable copy of the model before first use. Start with the test model described in test_plan.md to verify the scripts in your Enterprise Studio version.
2. In Enterprise Studio, use the Query tool's script/viewpoint management to create a script viewpoint. Copy the complete contents of the matching `.script` file into it, give it a clear name, and save it in the model. The exact menu labels can vary by Enterprise Studio version; see Bizzdesign's Working with the Query tool help.
3. Set the viewpoint's output presentation type from the table in section 2. Bizzdesign describes these options in Script output presentation types.
4. Install `data_flow_path_engine.script` first. Its saved viewpoint name must be exactly `data_flow_path_engine`; the other analysis scripts call that name. It is a dependency, not a standalone report.
5. To install both table and highlight versions of lineage or impact analysis, create two viewpoints from the same source file. Leave `MODE = "table";` in the table copy. Change it to `MODE = "highlight";` in the other copy, and set that copy's presentation type to `View: Highlight view`.
6. For context-menu actions, configure the corresponding `*_contextmenu.script` as the action's enablement script, and associate the action with its main script. Enablement scripts return `true` or `false`; they are not reports. See the inventory for the matching pairs.

Use unique viewpoint names for the runnable scripts. Their names may be chosen to suit the model, except for the path engine, whose name is fixed. Keep the source files with the model's deployment materials so the viewpoints can be updated consistently later.

2. Script inventory and output types

Source file	Viewpoint / configuration	Output presentation	How to use
`data_flow_path_engine.script`	Name exactly `data_flow_path_engine`	Table; dependency only	Called by the analysis and association scripts. Do not run directly; it expects an input variable named `dataObject`.
`setup_test_project.script`	Any name	Table	Run only in a new, empty ArchiMate test model, with the `Seed` view selected. Builds the scenarios in test_plan.md; do not run in a production model.

Source file	Viewpoint / configuration	Output presentation	How to use
`associate_data_n_flow.script`	Any name	Table	Select exactly one Data Object. Adds associations for the engine's producer-to-consumer paths; it does not remove extra associations.
`data_flow_consistency_check.script`	Any name	Table	Select one or more Data Objects to check only those; with no Data Object selected it checks every Data Object in the model. Report only; it does not change the model.
`data_flow_consistency_fix.script`	Any name	Table	Select the Data Object(s) to fix. With nothing selected, it considers every Data Object in the model. Prompts before adding missing or deleting extra flow associations.
`data_lineage.script`	Table copy: `MODE = "table";`	Table	Select one or more Data Objects. Reports roles, owning components, and selected paths.
`data_lineage.script`	Highlight copy: `MODE = "highlight";`	View: Highlight view	Select one or more Data Objects. Highlights them and their functions, flows,

Source file	Viewpoint / configuration	Output presentation	How to use
			access relations, and association relations on the view.
`data_impact_analysis.script`	Table copy: `MODE = "table";`	Table	Select one or more Application Functions or Application Components. Reports impacted consumers, distinguishing `Lost` from `Degraded (alternative path exists)`.
`data_impact_analysis.script`	Highlight copy: `MODE = "highlight";`	View: Highlight view	Same selection as the table copy; highlights the selected scope, affected paths, consumers, and Data Objects on the view.
`routing_functions.script`	Any name	Table	Select Data Objects to limit the report; with no Data Object selected it checks all Data Objects. Lists path functions with no access relation to each Data Object.
`manage_data_roles.script`	Any name	Table	Select one Data Object, optionally with Application Functions to review. If no Data Object is selected, the script asks you to choose one. Changes access relations only;

Source file	Viewpoint / configuration	Output presentation	How to use
			run the consistency fix afterwards to reconcile flow associations.
`manage_data_roles_contextmenu.script`	Enablement script for `manage_data_roles.script`	Boolean condition, not a presentation	Enables the action for exactly one Data Object, optionally with Application Functions.
`create_access_relations.script`	Any name	Table	Select one or more Application Functions, optionally with an existing Data Object. The prompts let you create or choose a Data Object and set each function's access type.
`create_access_relations_contextmenu.script`	Enablement script for `create_access_relations.script`	Boolean condition, not a presentation	Enables the action when the selection contains only Application Functions and/or Data Objects. The main script still requires at least one Application Function.
`interface_report.script`	Any name	Table	Select one or more Application Components. Lists Flow relations that cross each selected component's boundary, including data associations.

Source file	Viewpoint / configuration	Output presentation	How to use
`interface_report_contextmenu.script`	Enablement script for `interface_report.script`	Boolean condition, not a presentation	Enables the action only when all selected items are Application Components.
`flow_query_table.script`	Any name	Table	Select one Flow relation. Lists the Data Objects associated with that flow.
`flow_query_table_contextmenu.script`	Enablement script for `flow_query_table.script`	Boolean condition, not a presentation	Enables the action only when all selected items are Flow relations.
`flow_data_tooltip.script`	Any name	View: Tooltip view	Activate the tooltip viewpoint on a view, then hover over a Flow relation to see its associated Data Object names.
`flow_data_report.script`	Any name	Table	Run with no selection to list each model Flow relation and its associated Data Object(s), including flows with no associated Data Object.
`probe_view_items.script`	Any name; optional diagnostic	Table	Select view items and run from the Query tool to inspect their item/reference types. Intended for troubleshooting, not routine analysis.

The engine, lineage, impact, and routing calculations use the model's functions and flows; selecting a view does not restrict those analyses to that view. In contrast, highlight and tooltip presentation is shown on a view.

3. Core workflow

Check and associate flows

1. Select the Data Object and run `data_flow_consistency_check.script`. Review issues such as missing or extra associations, missing producers or consumers, and role/path problems. Informational rows prefixed `Note:` are not counted as issues.
2. To add associations without removing anything, select one Data Object and run `associate_data_n_flow.script`.
3. To reconcile both missing and extra associations, select the intended Data Object(s) and run `data_flow_consistency_fix.script`. Review each confirmation carefully. Re-run the consistency check to verify the result.

Do not run the consistency fix with nothing selected unless you intend to review changes for every Data Object in the model. It can add and delete association relations after confirmation.

Change roles

Use `manage_data_roles.script` when a function's access to a Data Object changes. The access type defines its role: `write` is a producer, `read/write` is a processor, `read` is a consumer, and `none` excludes that function from carrying the Data Object. A function without an access relation can be treated as a router on a path.

The script reviews current consumers and explicitly selected functions, and may ask whether to include the consumers' immediate Flow neighbours. If a current consumer is explicitly selected, only the explicitly selected functions are reviewed and there is no neighbour question. Other roles remain unchanged unless selected.

The script gathers answers before applying changes; cancelling during the questions leaves the model unchanged. It edits Access relations, not Flow or Association relations. After confirming role changes, run the consistency check and then the consistency fix for the affected Data Object.

Use `create_access_relations.script` when assigning access from selected functions to a new or existing Data Object. Unlike role management, it starts from the selected functions and creates or updates their Access relations.

Read reports

- Use `data_lineage.script` to see the roles and the selected producer-to-consumer paths for Data Objects.
- Use `data_impact_analysis.script` to assess the downstream impact of selected functions or components. A component includes its assigned functions and nested components/functions.
- Use `routing_functions.script` to review functions on paths that have no Access relation to the Data Object.
- Use `interface_report.script` to inspect incoming and outgoing flows across selected component boundaries.
- Use `flow_query_table.script`, `flow_data_tooltip.script`, or `flow_data_report.script` to inspect flow-to-Data-Object associations at one flow, on a view, or model-wide, respectively.

4. Interpretation and safety notes

- Path analysis uses Application Functions, Access relations, and Flow relations. It is model-wide, not limited to the currently open view.
- A consumer (`read`) ends a path. If that function forwards the Data Object, model it as `read/write` instead.
- Several producers are allowed. The engine analyzes each producer separately and selects paths per producer-consumer pair. Where a processor-bearing path exists, a path that bypasses its processors may be excluded; otherwise shortest paths are retained, including equal-length alternatives.
- Access type `none` excludes a function from transit. It is different from having no Access relation, which can make a function a router.
- `associate_data_n_flow.script` only adds associations. `data_flow_consistency_fix.script` may add or delete associations, and `manage_data_roles.script` / `create_access_relations.script` alter access modeling. Review selected objects and confirmation prompts before proceeding.

- The scripts report metamodel restrictions where handled, but only Enterprise Studio execution verifies API compatibility, permissions, relation creation, and view rendering for your model and version.

5. References

- Project scenarios and expected results: test_plan.md
- Bizzdesign Help: Working with the Query tool
- Bizzdesign Help: Script output presentation types
- Bizzdesign Help: Examples of using scripts in Enterprise Studio